

No Spoiler Please! A TinyBERT Model to Detect Spoiler Reviews on IMDb

Marco Colangelo^{1*}

¹Department of Computer Science, University of Milan, Milan, Italy.

Corresponding author(s). E-mail(s): marco.colangelo@studenti.unimi.it;

Abstract

This report contains the explanation of the work done for the final project of the Natural Language Processing course. The professor in charge of the course is Professor Alfio Ferrara, University of Milan. The project focused on the application of knowledge distillation, a technique used to compress large language models into smaller and more efficient versions. This is achieved by training a smaller model (the student) to replicate the behavior of a larger model (the teacher). In particular, this project had the aim to implement a spoiler detector for IMDb's movie reviews: a BERT-base model has been used as the teacher model, while a TinyBERT-General-4L-312D has been used as the student model. The experiments show that WAITING FOR RESULTS

1 Introduction

Very often, before even seeing a film, we decide to read some reviews because we're curious to know the opinions of those who have already seen it. In such a situation, it's not uncommon to encounter reviews that, whether intended to spoil something or not, include spoilers (e.g., "The movie is great, but character X dies/does this or that"). Clearly, we want to avoid this: luckily, platforms like IMDb warn us in advance about such reviews.

IMDb is by far the most popular global source for movies and TV shows, and it also includes user-submitted reviews with spoiler warnings for each one. The problem is that this approach isn't automated: users label reviews as spoilers themselves, but they aren't required to do so. The result is that spoiler reviews can exist without warnings as such (users can flag a review as a spoiler, and the IMDb moderation team will label it as such, but it's not immediate). What if there were a browser extension

(running locally on users' PCs) that retrieves the text of a review and tells us "Spoiler alert!" before we can even begin reading it? With the recent evolution of LLMs, this task can be accomplished by fine-tuning a sufficiently large pre-trained model, such as BERT.

Language model pre-training, such as BERT, has significantly improved the performances of many NLP tasks. However, pre-trained language models are usually computationally expensive, so it is difficult to efficiently execute them on resource-restricted devices (such as the user's PC). A novel Transformer distillation method, specially designed for knowledge distillation of the Transformer-based models, allows to effectively transfer to a smaller student model called TinyBERT the plenty of knowledge from a large pre-trained BERT model. A two-stage learning framework for TinyBERT which performs Transformer distillation at both the pretraining and task-specific learning stages ensures that TinyBERT can capture the general-domain as well as the task-specific knowledge in BERT. [1]

1.1 Related Work

So far, three main studies have defined the paradigm of model compression via knowledge transfer. The first, by Hinton et al. [2], introduced the seminal concept of Knowledge Distillation (KD), focusing on transferring the dark knowledge from a large Teacher to a compact Student. Their approach minimizes the distance between the output probability distributions of the two networks, softened by a temperature T . The second and more specialized study, by Jiao et al. [1], proposes a variant named TinyBERT, specifically designed for Transformer-based architectures. While Hinton's method operates on the final output layer, Jiao et al. argue that for complex language models, this is insufficient. Their method leverages a layer-to-layer distillation strategy, forcing the student to mimic not only the teacher's prediction but also its intermediate representations. The third, by Ji & Zhu [3], attempts to provide a mathematical justification for why knowledge distillation works, demonstrating that the use of hard labels (ground truth) are necessary during distillation to counteract the so-called "imperfect teacher" (i.e., a teacher who makes incorrect predictions). This way, the student can understand how much he can trust the teacher's predictions.

2 Research question and methodology

This work aims to implement Task Specific Distillation [1] to obtain a small BERT-based model specialized for spoiler detection. The novelty of this work is the use of the imperfect teacher concept [3] to implement the prediction layer loss.

Devo fare un progetto di nlp, che consiste nel fine tunare un modello BERT (nello specifico bert-base-uncased) sul gioco finale de avanti un altro! su canale 5 (do per scontato che tu sappia in cosa consiste :)). Il problema maggiore di questo progetto sta nel trovare un dataset o una qualunque altra fonte di dati da usare per il fine tuning, che si ridurrà ad una binary text classification (data una domanda del gioco con le due possibili risposte, il modello deve predire la risposta corretta, ossia quella sbagliata). Ti chiedo di trovare anzitutto dei lavori in cui è stata trovata una fonte dati sufficientemente grande (o direttamente dataset / fonti di dati da siti web / altro).

Successivamente, sarebbe utile sapere se qualcun'altro ha già fine tunato BERT su questo task, per avere un benchmark di paragone.

2.1 Data

The dataset used was the IMDB Spoiler Dataset [4], available on Kaggle. This dataset (in JSON format) contains over 500,000 IMDb movie reviews, each with a boolean label that marks it as a spoiler or not.

2.1.1 Preprocessing

Given the size of the dataset, I chose to extract a sample of 300,000 total reviews, then divided them into a training set (80%), validation set, and test set (10% each).

To overcome the 512 maximum token limit with BERT, a solution based on the head+tail of the review text was used [5]: the text is first tokenized using the BERT vocabulary, also adding the special tokens [CLS] at the start and [SEP] at the end. If the resulting token list is longer than 512, only the first 128 tokens after [CLS] and the last 382 before [SEP] are retained.

Each row, saved in a .parquet file for efficiency, contains the list of 512 tokens, an attention_mask of length 512 to highlight to the model that all tokens are important, and the label 0/1.

2.2 Implementation

2.2.1 Structure of the models

The choosen teacher model was BERT-base-uncased [6], while the choosen student model was TinyBERT.General.4L.312D [1]: this because this is exactly the TinyBERT model from Jiao et al with General Knowledge Distillation already performed (and therefore not necessary for my work) and, more importantly, it allows to implement the same techniques they use to handle the mismatch in the number of layers and dimensions of hidden states and embeddings [1]. Table 1 shows the differences in the number of parameters between the two models.

Table 1 Parameters of the teacher and student models.

Model	Layers	Attention Heads	Embeddings	Hidden States	Total Parameters
BERT-base-uncased	12	12	768	768	110M
TinyBERT.General.4L.312D	4	12	312	312	14.5M

2.2.2 BERT Fine-tuning

Since the dataset contains an imbalance in labels (73% False and 26% True), a weighted cross entropy loss was used during BERT fine-tuning, giving more importance to the

minority class (True) to help the model learn to recognize spoilers better. The weight has been calculated on the train set as the ratio between the number of negative samples and the number of positive samples, resulting in a weight of approximately 2.8 for the positive class.

The BERT teacher model was fine-tuned on the training set for 3 epochs, with a batch size of 24 (physical dimension = 8 with gradient accumulation = 3), a warmup ratio of 0.06, weight decay of 0.01, and AdamW as the optimizer. For learning rate, I applied a Layer-wise Decreasing Learning Rate strategy: a base LR of $2e-5$ for the top layer, with a decay factor of 0.95 to assign a lower weight to the bottom layer. The metric chosen to select the best model was PR-AUC, due to the imbalance of the dataset.

Figure DA INSERT shows the metrics obtained with the fine-tuned teacher compared to the teacher without fine-tuning.

2.2.3 Task Specific Distillation

The task-specific distillation phase is divided into two distinct phases.

- Phase 1: Transformer and Embedding Distillation.
- Phase 2: Prediction Layer Distillation

3 Results

4 Conclusion

References

- [1] Jiao, X., Yin, Y., Shang, L., Jiang, X., Chen, X., Li, L., Wang, F., Liu, Q.: TinyBERT: Distilling BERT for Natural Language Understanding (2020). <https://arxiv.org/abs/1909.10351>
- [2] Hinton, G., Vinyals, O., Dean, J.: Distilling the Knowledge in a Neural Network (2015). <https://arxiv.org/abs/1503.02531>
- [3] Ji, G., Zhu, Z.: Knowledge Distillation in Wide Neural Networks: Risk Bound, Data Efficiency and Imperfect Teacher (2020). <https://arxiv.org/abs/2010.10090>
- [4] Misra, R.: IMDB Spoiler Dataset. <https://doi.org/10.13140/RG.2.2.11584.15362>
- [5] Sun, C., Qiu, X., Xu, Y., Huang, X.: How to Fine-Tune BERT for Text Classification? (2020). <https://arxiv.org/abs/1905.05583>
- [6] Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. CoRR **abs/1810.04805** (2018) [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)